Homework Set 3 – Virtual Memory, I/O, File Systems

Due Date: Monday, June 1, 2026

Note: In this homework set, when counting capacity, e.g., bytes and bits, 1k = 2^{10}, 1M = 2^{20}, and 1G = 2^{30}.

Problem 1: Virtual Memory. [25 points]

1. Consider a virtual memory scheme that uses multi-level page tables, with the following format for virtual addresses:

Virtual Page # (10 bits)	Virtual Page # (10 bits)	Offset (12 bits)

That means, the first-level page table uses the highest 10 bits of virtual address. The second-level page table uses the next 10 bits of virtual address.

Virtual addresses are translated into physical addresses of the following form:

Physical Page # (20 bits)	Offset (12 bits)

Page table entries (PTE) are 32 bits in the following format, stored in *big-endian form* (i.e. the most significant byte has the lowest address) in memory

Physical Page # (20 bits)	OS Defined (3 bits)	0	Large Page	Dirty	Accessed	Nocache	Write Through	User	Writeable	Valid

Here, “Valid” means that a translation is valid; “Writeable” means that the page is writeable; “User” means that the page is accessible by the user (rather than only by the Kernel). Note: the phrase “page table” in the following questions means the entire multi-level data structure that maps virtual addresses to physical addresses.

1. [2 points] How big is a page?

2. [2 points] What is the maximum size of physical memory this system can support?

3. [6 points] For this scheme, what is the maximum size that a page table of a process can be (in bytes)?

4. [3 points] Suppose that a process has an address space with one page at the top of the address space and one page at the bottom of the address space. How big would the page table be (in bytes)?

5. [7 points] Assume a process uses 512 kB of physical memory. What is the minimum size of its page table (in bytes)? What is the maximum size of its page table (in bytes)?

6. [5 points] Now we would like to add a fully-associative TLB into this system to make address translation faster. Recall that each TLB entry treats the PTE as the data, and also needs a tag and a valid bit. How many bits would each entry of such a fully-associative TLB have? Show your progress to reach the answer.

Problem 2: Paging. [15 points]

A main memory consists of four physical pages (frames), denoted as i, ii, iii, iv. Assume the LRU page replacement algorithm, and initially all physical frames are empty.

Consider the following access stream of virtual pages A, B, C, D, E, C, A, B, C, D, F.

1. [12 points] Fill in the following table showing all page faults. In each column, if a page fault occurs for this access, fill the virtual page name (e.g., A, B) in the cell of the row that corresponds to the physical frame (e.g., i, ii) to which this page will be paged to.

What is the total number of page faults?

	A	B	C	D	E	C	A	B	C	D	F
i	A										
ii											
iii											
iv											

2. [3 points] What is the minimum memory size (in number of physical frames) such that the number of faults to be no larger than 6? Explain.

Problem 3: Disks. [20 points]

Average and minimum times for reading and writing to storage devices are common measurements used to compare devices. Calculate values related to read and write times for disks with the following characteristics. These latencies start from the access request is issued to the device controller, until the data transfer is completed. Disk transfer and controller transfer are not pipelined.

	Average seek time	RPM	Disk transfer rate	Controller transfer rate
a.	11ms	7200	36 MBytes/s	500 Mbits/s
b.	9ms	7200	32 MBytes/s	520 Mbits/s

1. [10 points] Calculate the average time to read or write a 1024 byte sector for each disk listed in the table.

2. [6 points] Calculate the minimum time to read or write a 2048 byte sector for each disk listed in the table.

3. [4 points] For each disk in the table, determine the dominant factor for performance. Specifically, if you could make an improvement to any aspect of the disk, what would you choose? If there is no dominant factor, explain why. Hint: consider different access patterns.

Problem 4: I/O Notification. [10 points]

An important advantage of interrupts over polling is the ability of the processor to perform other tasks while waiting for communication from an I/O device. Suppose that a 1 GHz processor needs to read 1000 bytes of data from a particular I/O device. The I/O device supplies 1 byte of data every 0.02 ms. The code to process each byte of data after receiving and store it in a buffer takes 100 cycles.

1. [5 points] If the processor detects whether a byte of data is ready by keeping polling without sleep, and one poll takes 50 cycles, how many cycles does the entire operation (i.e., reading all 1000 bytes) take? How many polls will the processor do in total? Assume that only after one byte arrives can one poll successfully find the byte, and the first byte arrives right before the first poll.

2. [5 points] If instead, the processor is interrupted when a byte is ready, and the processor spends the time between interrupts on another task, how many cycles of this other task can the processor complete while the I/O communication is taking place? The overhead for handling an interrupt is 200 cycles. During each interrupt, we process the byte and store it in a buffer as mentioned above.

Problem 5: File Systems. [30 points]

Consider a UNIX file system. Disk blocks are 4096 bytes. Sectors are 512 bytes long. All block pointers are 32-bit long. An inode has 12 direct block pointers, one indirect block pointer, and one double-indirect block pointer. Both indirect and double-indirect blocks take up an entire disk block. The total inode size is 256 bytes.

1. [8 points] How much disk space, including metadata and data blocks, is needed to store a 4 GB DVD image file? Show your calculation.

Give your answer in bytes. When accounting for the inode, because multiple inodes can share a single block, we only count 256 bytes rather than a full block.

2. [3 points] What is the maximum file size (in bytes) that can be stored in this file system?

3. [6 points] Assume that there is no buffer cache, i.e., no filesystem structures or data are cached by OS in main memory. starting from the i-number, how many disk accesses will be required to read only the last byte in this file? How about to overwrite it? Show your calculations. Each access can get an entire disk block.

4. [5 points] Suppose that among all files in the file system, half of them are 3 kB and the other half are 8 kB. Considering only disk space used to store data and ignoring metadata and directory structures, what is the space overhead, in terms of the ratio of allocated space vs. actual data size? How about on a disk with 2 kB block size?

5. [4 points] In UNIX-like operating systems, the command `mv` can move a file from one directory to another. For example, "`mv /a/b/c /d/e/f`". Please explain how many blocks are changed in this `mv` command, and what are they? You do not need to consider timestamp changes in this question.

6. [4 points] Consider the following set of file system metadata.

Directory, file metadata (e.g., inode + (double-)indirect blocks), free map

Which one(s) need to be changed to add link supports in a file system? How?

1) To support soft (symbolic) links.

2) To support hard links.